

LLM Powered Autonomous Agents

Date: June 23, 2023 | **Author:** Lilian Weng

Source: Lil'Log (<https://lilianweng.github.io/posts/2023-06-23-agent/>)

Building agents with LLM (large language model) as its core controller is a cool concept. Several proof-of-concepts demos, such as AutoGPT, GPT-Engineer and BabyAGI, serve as inspiring examples. The potentiality of LLM extends beyond generating well-written copies, stories, essays and programs; it can be framed as a powerful general problem solver.

Agent System Overview

In a LLM-powered autonomous agent system, LLM functions as the agent's brain, complemented by several key components:

- **Planning:**

- Subgoal and decomposition: The agent breaks down large tasks into smaller, manageable subgoals, enabling efficient handling of complex tasks.
- Reflection and refinement: The agent can do self-criticism and self-reflection over past actions, learn from mistakes and refine them for future steps, thereby improving the quality of final results.

- **Memory:**

- Short-term memory: Utilizes all in-context learning through prompt engineering methods to steer behaviors.
- Long-term memory: Retains and recalls infinite information over extended periods, often leveraging an external vector store and fast retrieval.

- **Tool use:** The agent learns to call external APIs for extra information that is missing from the model weights (often hard to change after pre-training), including current information, code execution capability, access to proprietary information sources and more.

Component One: Planning

A complicated task usually involves many steps. An agent needs to know what they are and plan ahead.

Task Decomposition

Chain of Thought (CoT; Wei et al. 2022) has become a standard prompting technique for enhancing model performance on complex tasks. The model is instructed to “think step by step” to utilize more test-time computation to decompose hard tasks into smaller and simpler steps. CoT transforms big tasks into multiple manageable tasks and sheds light on an interpretation of the model's thinking process.

Tree of Thoughts (Yao et al. 2023) extends CoT by exploring multiple reasoning possibilities at each step. It first decomposes the problem into multiple thought steps and generates multiple thoughts per step, creating a tree structure. The search process can be BFS (breadth-first search) or DFS (depth-first search) with classifiers evaluating states via LLM prompts.

Another approach is using external classic planners. For example, **LLM+P (Liu et al. 2023)** involves an external PDDL (Planning Domain Definition Language) planner to map out action structures, letting the LLM translate natural language problems into PDDL inputs and back.

Self-Reflection

Self-reflection is a vital aspect that allows autonomous agents to improve iteratively by refining past action decisions and correcting previous mistakes. It plays a crucial role in real-world tasks where trial and error are inevitable.

- **ReAct (Yao et al. 2023):** Integrates reasoning and acting within LLM by extending the action space to be a combination of task-specific discrete actions and the language space. The language thoughts help the model formulate plans, execute actions, and continuously adapt to environment observations.
- **Reflexion (Shinn & Labash 2023):** Formulates the agent with explicit memory of verbal self-reflections. The environment provides a binary or numeric score, which pushes a specialized evaluator LLM to generate feedback. This reflection is preserved as context for the next episode, allowing the agent to systematically avoid repeating structural mistakes.
- **Chain of Hindsight (CoH; Liu et al. 2023):** Trains models sequentially using human alignment data. Instead of just reinforcing positive paths, it presents feedback as natural text sequences, teaching the model to dynamically update outputs based on explicit critique.

Component Two: Memory

Memory can be structurally compared to the human cognitive system for easy mapping.

1. **Sensory Memory:** Captures structural inputs directly (like raw visual/audio patches or context-window bursts), providing immediate frame reflection lasting only a few seconds.

2. **Short-Term Memory / Working Memory:** Consists of in-context prompts and operational history buffers. Its capacity is constrained by the strict token length limitation of the host language model.
3. **Long-Term Memory:** Acts as an external structural archive. It maps information using high-dimensional text embeddings into a Vector Database, matching relevant patterns via Maximum Inner Product Search (MIPS).

Given a query vector q and a set of document vectors V , the goal is to find:

$$v^* = \arg \max_{v \in V} (q \cdot v)$$

Common MIPS Vector Indexing Algorithms

Algorithm Type	Core Mechanism	Key Advantages
LSH (Locality-Sensitive Hashing)	Applies hashing functions to group close points into the same buckets.	Highly efficient lookup speeds; sub-linear search time.
HNSW (Hierarchical Navigable Small World)	Builds a multi-layer graph structure mirroring complex internet topologies.	Exceptional recall accuracy; scales well to high-dimensional spaces.
Inverted File Index (IVF)	Clusters space partitions using k-means, querying only central proximity indices.	Low memory footprints; ideal for extreme-scale text indexing.

Component Three: Tool Use

Humans utilize external physical and analytical tools to extend their sensory and cognitive boundaries. Similarly, equipping LLMs with external APIs, database extensions, and localized programmatic engines radically boosts their accuracy and consistency.

- **MRKL (Modular Reasoning, Knowledge and Language; Karpas et al. 2022):** An architecture that pairs an LLM with external symbolic modules (like calculators or database routers). The LLM recognizes when to route arithmetic or data queries away from its weights to precise, predictable code modules.
- **TALM (Tool Augmented Language Models; Parisi et al. 2022) & Toolformer (Schick et al. 2023):** Fine-tunes models to insert explicit API call tokens directly inside text sequences. The model proactively stops text generation to await external server evaluation, blending API query tokens naturally with regular text outputs.

- **HuggingGPT (Shen et al. 2023):** Employs ChatGPT as a centralized structural orchestrator to control hundreds of specialized open-weights models hosted on Hugging Face. The master model treats sibling models as standard functional tools to execute multi-modal pipelines covering video, speech, and image synthesis.

Case Studies & Implementation Landscapes

Scientific Discovery Agent

ChemCrow (Bran et al. 2023) serves as a robust real-world implementation where an LLM agent is provided with 17 domain-specific tools covering chemical literature search, molecule synthesis execution, and analytical lab safety suites. When prompted to synthesize a specific compound, the agent independently validates safety codes, plans intermediate synthesis paths, and coordinates robotic automation execution steps cleanly.

Generative Agents Simulation

Park et al. (2023) deployed a virtual sandbox community containing 25 distinct generative agents acting out structured human lives. Each agent possessed an advanced architecture interacting with a local environment, processing a dynamic reflection loop:

- **Retrieve:** Pulls relevant memories based on recency, importance, and situational context.
- **Reflect:** Periodically pauses events to synthesize raw observations into higher-level abstract ideas.
- **Plan:** Translates abstract mental goals into granular timeline itineraries, adapting on-the-fly when interacting with other agents or objects.

Key Challenges Ahead

Despite rapid structural evolution, several critical bottlenecks remain across LLM agent architectures:

1. **Finite Context Constraints:** Long-term historical planning is heavily bounded by active context limits. While modern architectures support massive expansion windows, parsing thousands of previous token paths degrades retrieval precision.
2. **Reliability of Tool Integration:** Agents frequently mismatch input parameters when formulating raw API calls, failing to recover when unexpected network or software errors emerge.
3. **Compounding Error Loops:** If early planning phases output hallucinated facts or invalid tool responses, subsequent self-reflection loops can amplify these biases, trapping the agent in recursive, unproductive execution states.